

Отчет по лабораторной работе 3

Веб сервис Asteroid Dates для SPA

Параметр	Значение
Студент	Дарья Просвиракова
Проект	Asteroid Dates
Backend	NestJS, TypeORM, PostgreSQL
Текущий пользователь в лабораторной	id=1, Дарья Просвиракова, daria@example.com
Дата проверки	6 октября 2026

Цель работы: создать REST веб сервис на backend для использования из SPA, подключить его к базе данных, реализовать итоговую бизнес логику без полноценной авторизации и проверить методы через Postman.

В лабораторной работе реализованы домены услуг и пользователей. Системные поля не передаются клиентом для изменения: id, статусы, пользователь, даты и вычисляемые признаки задаются backend-логикой.

HTTP методы

Метод	URL	Назначение
GET	/api/dates?maxMonth=7	Список опубликованных услуг с фильтром по месяцу.
POST	/api/dates	Создание черновика. multipart/form-data: designation, image, video.
GET	/api/dates/draft	Получение текущего черновика пользователя без передачи id.
PUT	/api/dates/publish	Публикация текущего черновика. Статус меняется только backend-логикой.
GET	/api/dates/feed	Первая запись ленты опубликованных услуг.
GET	/api/dates/feed?id=2&next=true	Следующая запись ленты через запрос к БД, id может иметь пропуски.
POST	/api/dates/2/like	like=1 ставит лайк, like=0 отменяет. Количество в ответе ограничено 5.
DELETE	/api/dates/:id	Soft delete только своей услуги.

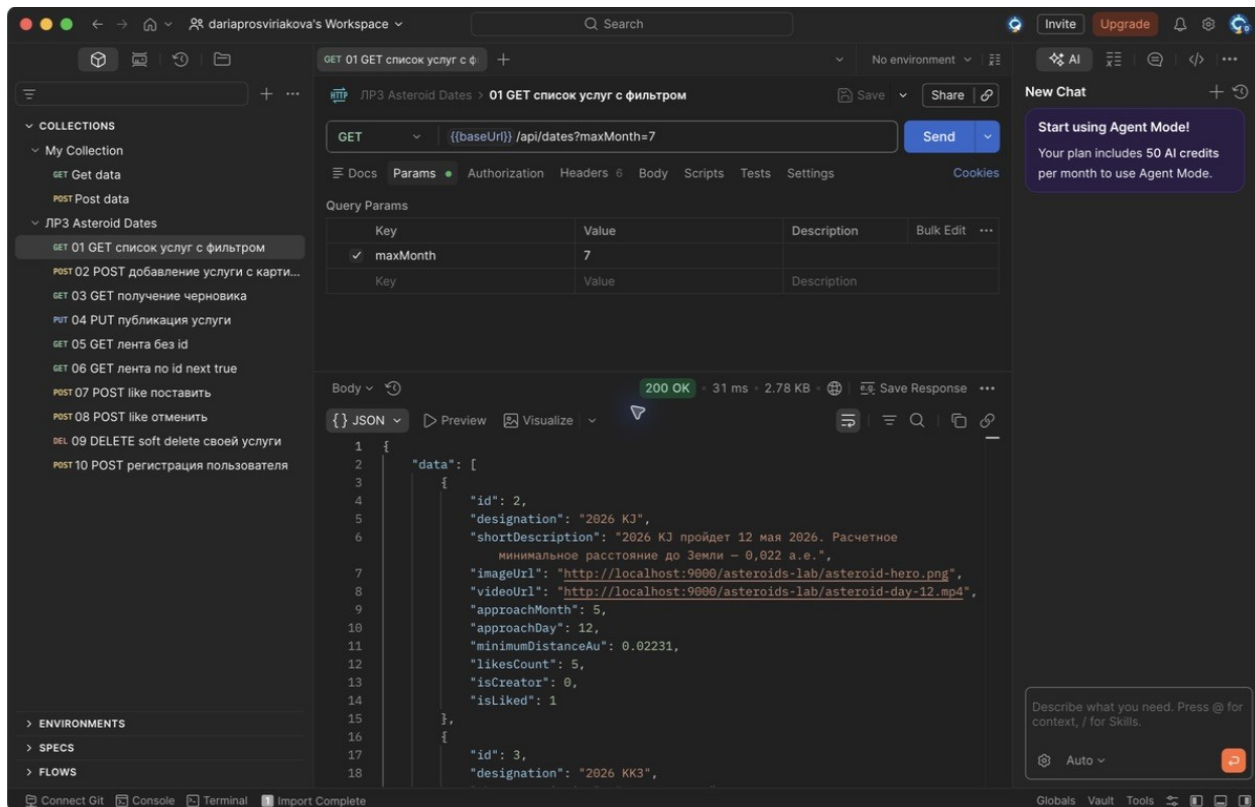
POST	/api/users/register	Регистрация пользователя.
POST	/api/users/auth, /api/users/logout	Заглушки аутентификации и деавторизации для следующей лабораторной.

Таблицы базы данных

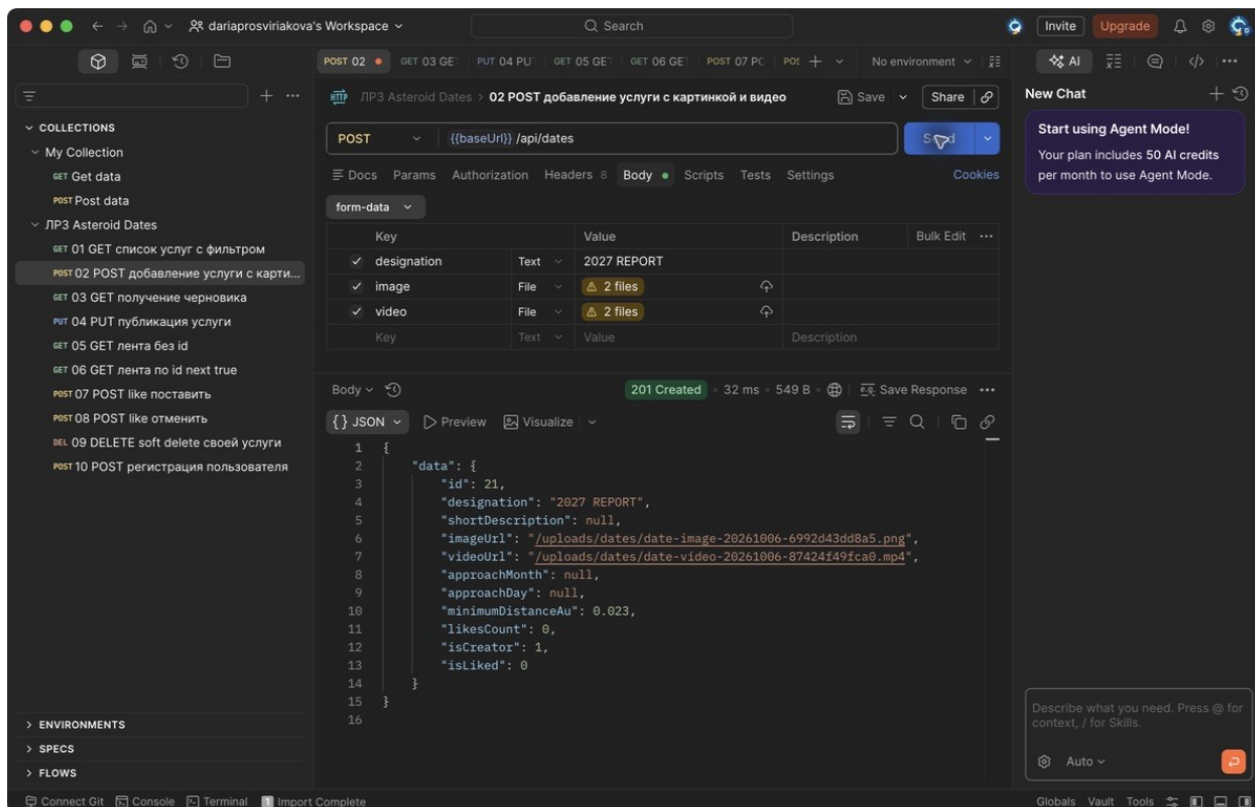
Таблица	Поля	Назначение
asteroid_dates	id, designation, short_description, status, image_url, video_url, created_at, formed_at, creator_id, approach_month, approach_day, minimum_distance_au	Услуги/даты сближения. Статусы: draft, published, deleted.
observers	id, full_name, email	Пользователи системы.
observer_date_likes	id, observer_id, asteroid_date_id	Связь пользователя и понравившейся услуги; уникальность observer_id + asteroid_date_id.

Демонстрация запросов Postman

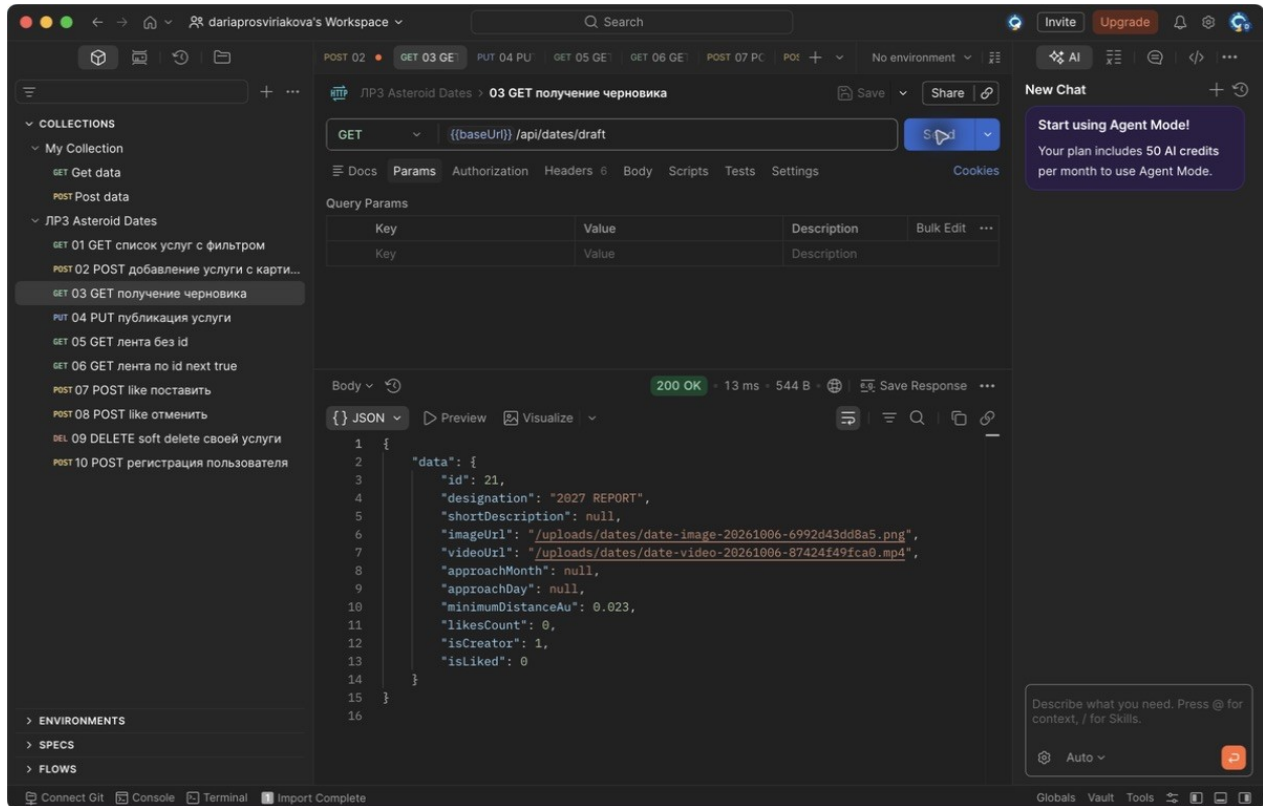
Скриншот 1. GET /api/dates?maxMonth=7, список опубликованных услуг с фильтром.



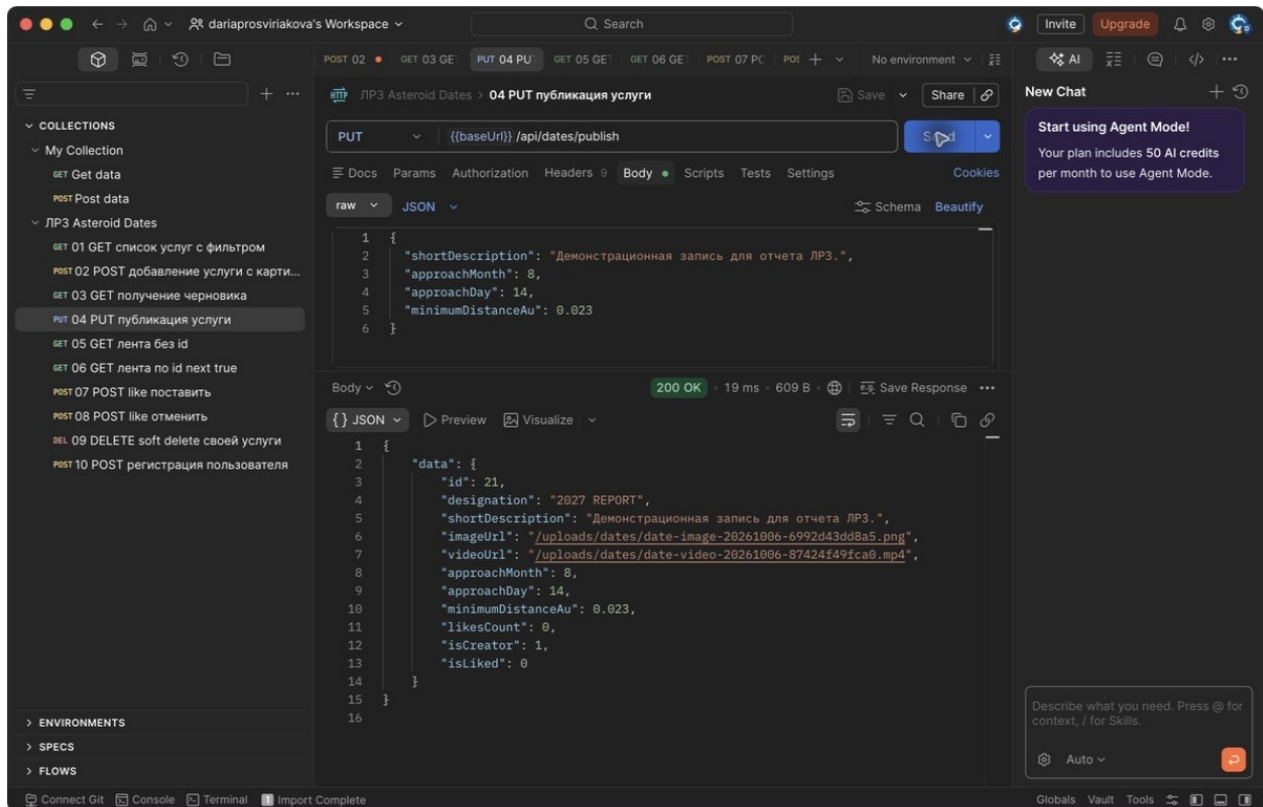
Скриншот 2. POST /api/dates, добавление черновика с image и video как файлами.



Скриншот 3. GET /api/dates/draft, получение текущего черновика.



Скриншот 4. PUT /api/dates/publish, публикация черновика.



Скриншот 5. GET /api/dates/feed, лента без id.

GET 01 GET cн POST 02 ПО GET 03 GET no PUT 04 PUT ny GET 05 GET лент без id

GET /api/dates/feed

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body

200 OK • 20 ms • 676 B

```
{
  "data": [
    {
      "id": 2,
      "designation": "2026 KJ",
      "shortDescription": "2026 KJ пройдет 12 мая 2026. Расчетное минимальное расстояние до Земли - 0,022 а.е.",
      "imageUrl": "http://localhost:9000/asteroids-lab/asteroid-hero.png",
      "videoUrl": "http://localhost:9000/asteroids-lab/asteroid-day-12.mp4",
      "approachMonth": 5,
      "approachDay": 12,
      "minimumDistanceAu": 0.02231,
      "likesCount": 5,
      "isCreator": 0,
      "isLiked": 1
    }
  ]
}
```

Скриншот 6. GET /api/dates/feed?id=2&next=true, следующая запись ленты.

GET 01 GET POST 02 F GET 03 GET PUT 04 PUT GET 05 GET GET 06 GET лент no id next true

GET /api/dates/feed?id=2&next=true

Query Params

Key	Value	Description	Bulk Edit
id	2		
next	true		
Key	Value	Description	

Body

200 OK • 20 ms • 680 B

```
{
  "data": [
    {
      "id": 3,
      "designation": "2026 KK3",
      "shortDescription": "2026 KK3 пройдет 15 мая 2026. Расчетное минимальное расстояние до Земли - 0,043 а.е.",
      "imageUrl": "http://localhost:9000/asteroids-lab/asteroid-day-15.png",
      "videoUrl": "http://localhost:9000/asteroids-lab/asteroid-day-15.mp4",
      "approachMonth": 5,
      "approachDay": 15,
      "minimumDistanceAu": 0.04297,
      "likesCount": 5,
      "isCreator": 0,
      "isLiked": 1
    }
  ]
}
```

Скриншот 7. POST /api/dates/2/like, установка лайка like=1.

The screenshot shows a REST client interface with a sidebar on the left containing a list of collections and environments. The main panel displays a POST request to the endpoint `POST /api/dates/2/like`. The request body is a JSON object:

```
{ 1: { 2: "like": 1 3: } }
```

. The response is a JSON object:

```
{ 1: { 2: "data": { 3: "like": 1, 4: "date": { 5: "id": 2, 6: "designation": "2026 КЗ", 7: "shortDescription": "2026 КЗ пройдет 12 мая 2026. Расчетное минимальное расстояние до Земли - 0,022 а.е.", 8: "imageUrl": "http://localhost:9000/asteroids-lab/asteroid-hero.png", 9: "videoUrl": "http://localhost:9000/asteroids-lab/asteroid-day-12.mp4", 10: "approachMonth": 5, 11: "approachDay": 12, 12: "minimumDistanceAu": 0.02231, 13: "likesCount": 5, 14: "isCreator": 0, 15: "isLiked": 1 16: } 17: } 18: }
```

. The status bar indicates a 201 Created response with a 24 ms duration and 697 B of data.

Скриншот 8. POST /api/dates/2/like, отмена лайка like=0.

The screenshot shows the same REST client interface as the previous one, but with a POST request to the endpoint `POST /api/dates/2/like` where the request body is

```
{ 1: { 2: "like": 0 3: } }
```

. The response is a JSON object:

```
{ 1: { 2: "data": { 3: "like": 0, 4: "date": { 5: "id": 2, 6: "designation": "2026 КЗ", 7: "shortDescription": "2026 КЗ пройдет 12 мая 2026. Расчетное минимальное расстояние до Земли - 0,022 а.е.", 8: "imageUrl": "http://localhost:9000/asteroids-lab/asteroid-hero.png", 9: "videoUrl": "http://localhost:9000/asteroids-lab/asteroid-day-12.mp4", 10: "approachMonth": 5, 11: "approachDay": 12, 12: "minimumDistanceAu": 0.02231, 13: "likesCount": 4, 14: "isCreator": 0, 15: "isLiked": 0 16: } 17: } 18: }
```

. The status bar indicates a 201 Created response with a 31 ms duration and 697 B of data.

Скриншот 9. DELETE /api/dates/21, soft delete своей услуги.

The screenshot shows the Postman interface with a DELETE request to `http://localhost:3000/api/dates/21`. The response is a 200 OK status with a JSON body indicating a soft delete.

Request: DELETE `http://localhost:3000/api/dates/21`

Response: 200 OK • 18 ms • 267 B

JSON Body:

```
1 {
2   "data": {
3     "id": 21,
4     "isDeleted": 1
5   }
6 }
```

Скриншот 10. POST /api/users/register, регистрация нового пользователя.

The screenshot shows the Postman interface with a POST request to `{{baseUrl}}/api/users/register`. The response is a 201 Created status with a JSON body indicating successful user registration.

Request: POST `{{baseUrl}}/api/users/register`

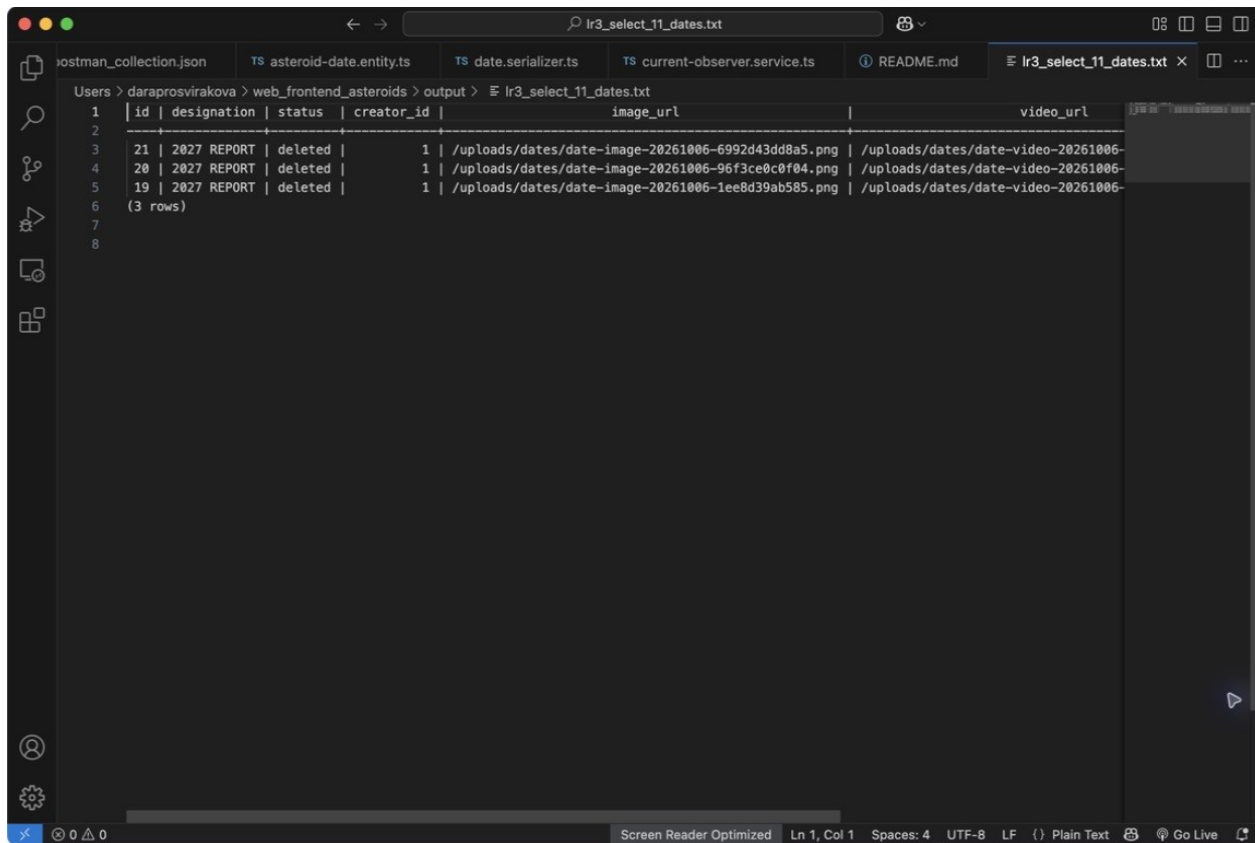
Response: 201 Created • 18 ms • 345 B

JSON Body:

```
1 {
2   "data": {
3     "id": 143,
4     "fullName": "Пользователь отчета",
5     "email": "report-demo@example.com"
6   }
7 }
```

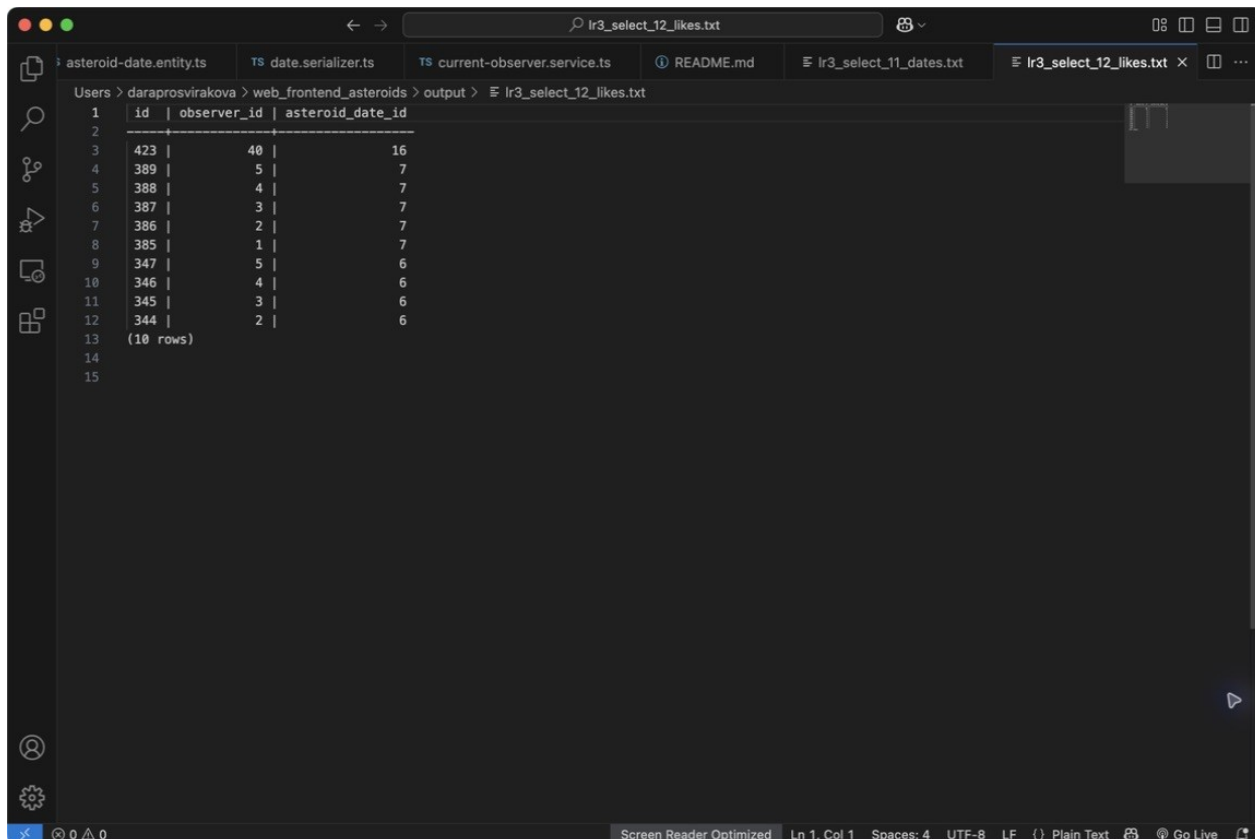
Проверка данных через SELECT

Скриншот 11. SELECT по asteroid_dates: созданные услуги переведены в deleted через soft delete.



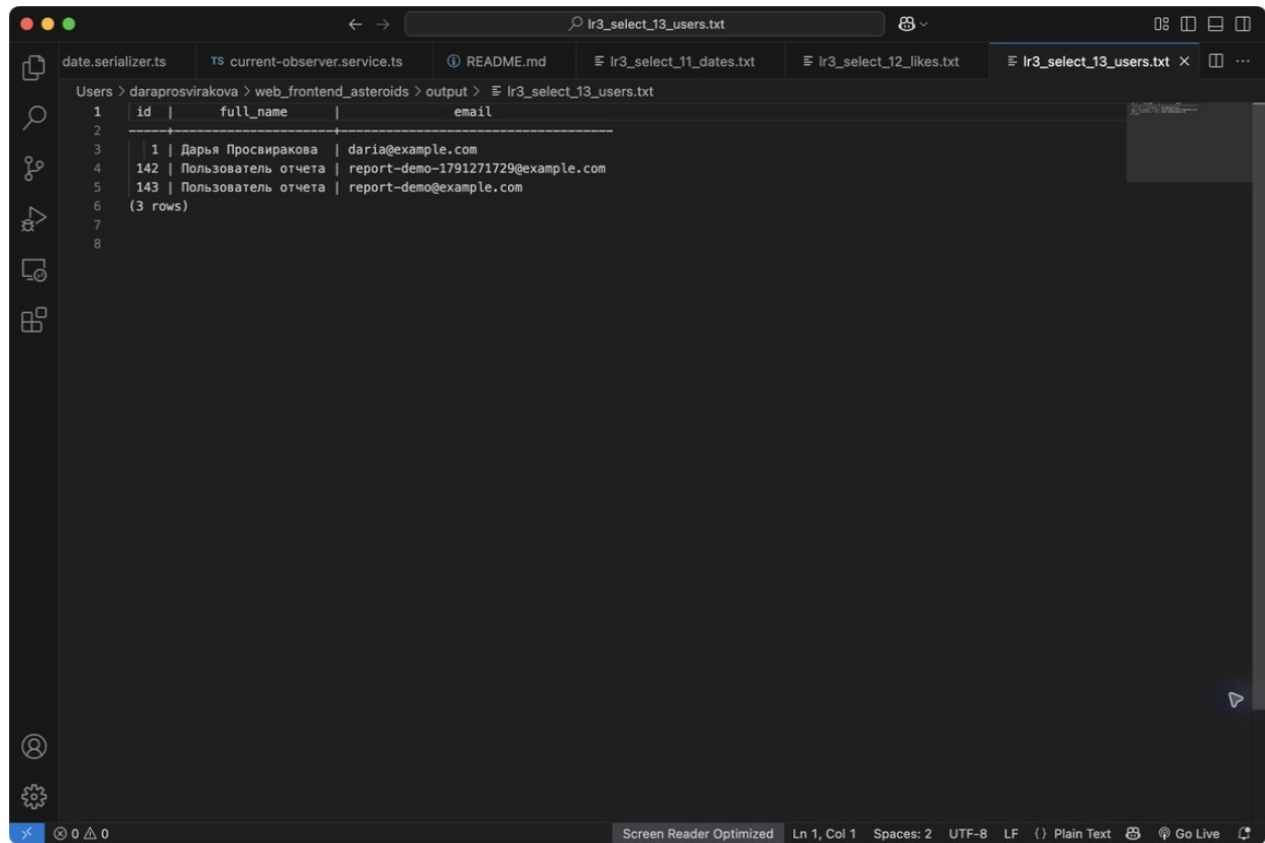
```
Users > daraprosvirakova > web_frontend_asteroids > output > lr3_select_11_dates.txt
1 | id | designation | status | creator_id | image_url | video_url |
2 |-----|-----|-----|-----|-----|-----|
3 | 21 | 2027 REPORT | deleted | 1 | /uploads/dates/date-image-20261006-6992d43dd8a5.png | /uploads/dates/date-video-20261006-6992d43dd8a5.png |
4 | 20 | 2027 REPORT | deleted | 1 | /uploads/dates/date-image-20261006-96f3ce0c0f04.png | /uploads/dates/date-video-20261006-96f3ce0c0f04.png |
5 | 19 | 2027 REPORT | deleted | 1 | /uploads/dates/date-image-20261006-1ee8d39ab585.png | /uploads/dates/date-video-20261006-1ee8d39ab585.png |
6 | (3 rows) |
7 |
8 |
```

Скриншот 12. SELECT по observer_date_likes: данные лайков после проверки методов.



```
Users > daraprosvirakova > web_frontend_asteroids > output > lr3_select_12_likes.txt
1 | id | observer_id | asteroid_date_id |
2 |-----|-----|-----|
3 | 423 | 40 | 16 |
4 | 389 | 5 | 7 |
5 | 388 | 4 | 7 |
6 | 387 | 3 | 7 |
7 | 386 | 2 | 7 |
8 | 385 | 1 | 7 |
9 | 347 | 5 | 6 |
10 | 346 | 4 | 6 |
11 | 345 | 3 | 6 |
12 | 344 | 2 | 6 |
13 | (10 rows) |
14 |
15 |
```

Скриншот 13. SELECT по observers: фиксированный пользователь и зарегистрированный пользователь отчета.



The screenshot shows a code editor window with a file named `lr3_select_13_users.txt` open. The editor displays the output of a SQL query, which is a table with three columns: `id`, `full_name`, and `email`. The table contains three rows of data. The first row has `id` 1, `full_name` Дарья Просвиракова, and `email` daria@example.com. The second row has `id` 142, `full_name` Пользователь отчета, and `email` report-demo-1791271729@example.com. The third row has `id` 143, `full_name` Пользователь отчета, and `email` report-demo@example.com. The table is preceded by a header row and a footer row indicating 3 rows. The editor's interface includes a sidebar with icons for Explorer, Search, Source Control, Run and Debug, and Extensions. The status bar at the bottom shows 'Screen Reader Optimized', 'Ln 1, Col 1', 'Spaces: 2', 'UTF-8', 'LF', and 'Plain Text'.

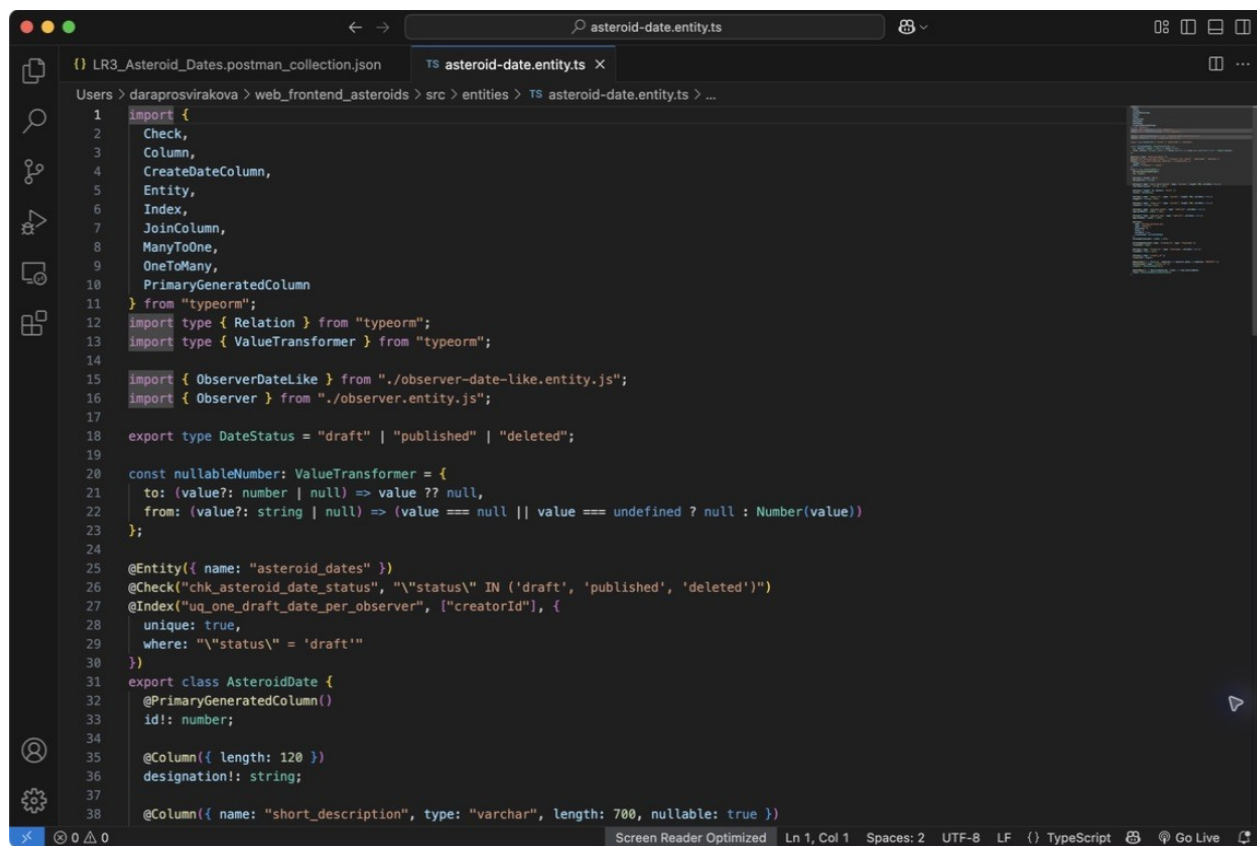
```
Users > daraprosvirakova > web_frontend_asteroids > output > lr3_select_13_users.txt
1  id | full_name | email
2  ---|-----|-----
3  1 | Дарья Просвиракова | daria@example.com
4  142 | Пользователь отчета | report-demo-1791271729@example.com
5  143 | Пользователь отчета | report-demo@example.com
6  (3 rows)
7
8
```

Модели и сериализаторы

Модель AsteroidDate описывает таблицу asteroid_dates, статусы, связи с пользователем и лайками.

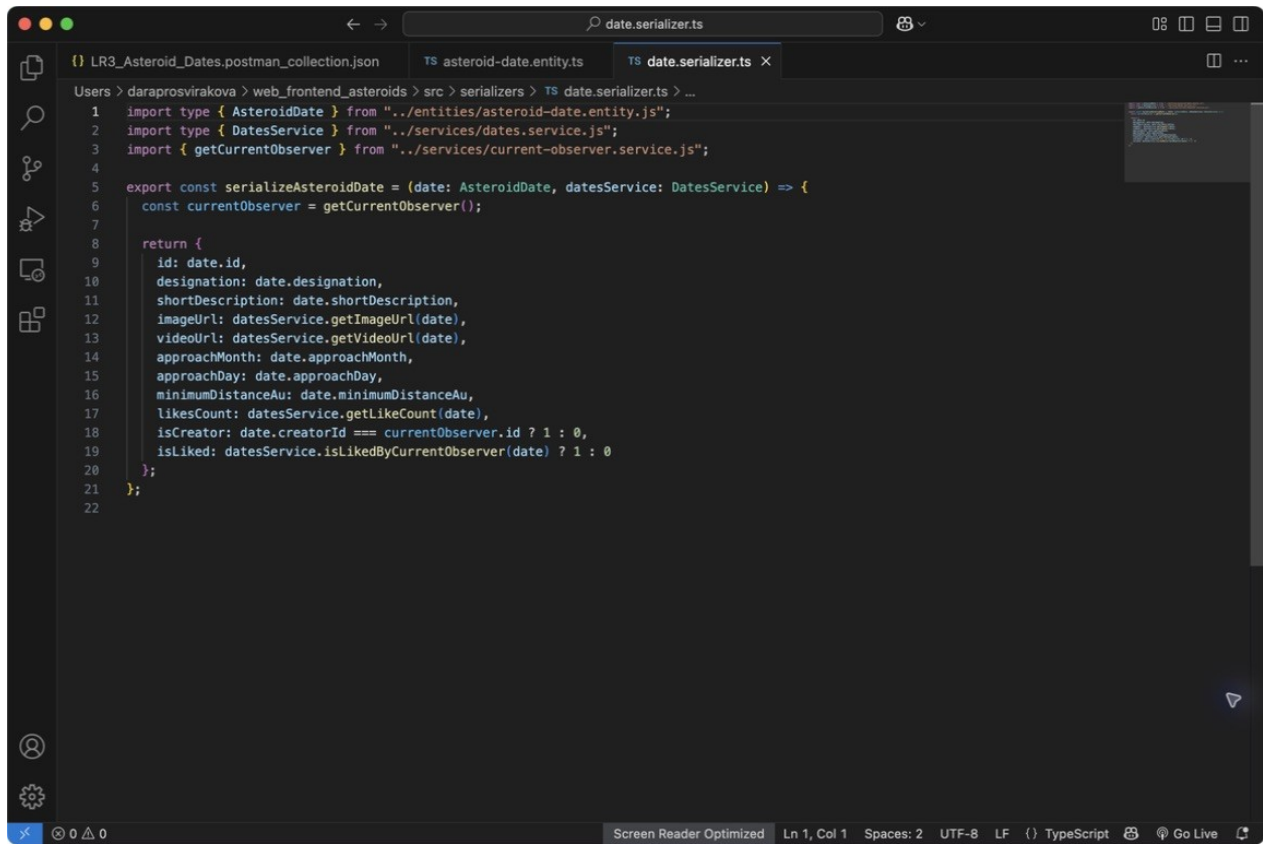
Сериализатор формирует ответ API и добавляет признаки isCreator и isLiked для текущего пользователя.

Скриншот 14. Модель AsteroidDate в TypeORM.



```
1 import {
2   Check,
3   Column,
4   CreateDateColumn,
5   Entity,
6   Index,
7   JoinColumn,
8   ManyToOne,
9   OneToMany,
10  PrimaryGeneratedColumn
11 } from "typeorm";
12 import type { Relation } from "typeorm";
13 import type { ValueTransformer } from "typeorm";
14
15 import { ObserverDateLike } from "../observer-date-like.entity.js";
16 import { Observer } from "../observer.entity.js";
17
18 export type DateStatus = "draft" | "published" | "deleted";
19
20 const nullableNumber: ValueTransformer = {
21   to: (value?: number | null) => value ?? null,
22   from: (value?: string | null) => (value === null || value === undefined ? null : Number(value))
23 };
24
25 @Entity({ name: "asteroid_dates" })
26 @Check("chk_asteroid_date_status", "\"status\" IN ('draft', 'published', 'deleted')")
27 @Index("uq_one_draft_date_per_observer", ["creatorId"], {
28   unique: true,
29   where: "\"status\" = 'draft'"
30 })
31 export class AsteroidDate {
32   @PrimaryGeneratedColumn()
33   id!: number;
34
35   @Column({ length: 120 })
36   designation!: string;
37
38   @Column({ name: "short_description", type: "varchar", length: 700, nullable: true })
```

Скриншот 15. Сериализатор ответа услуги.



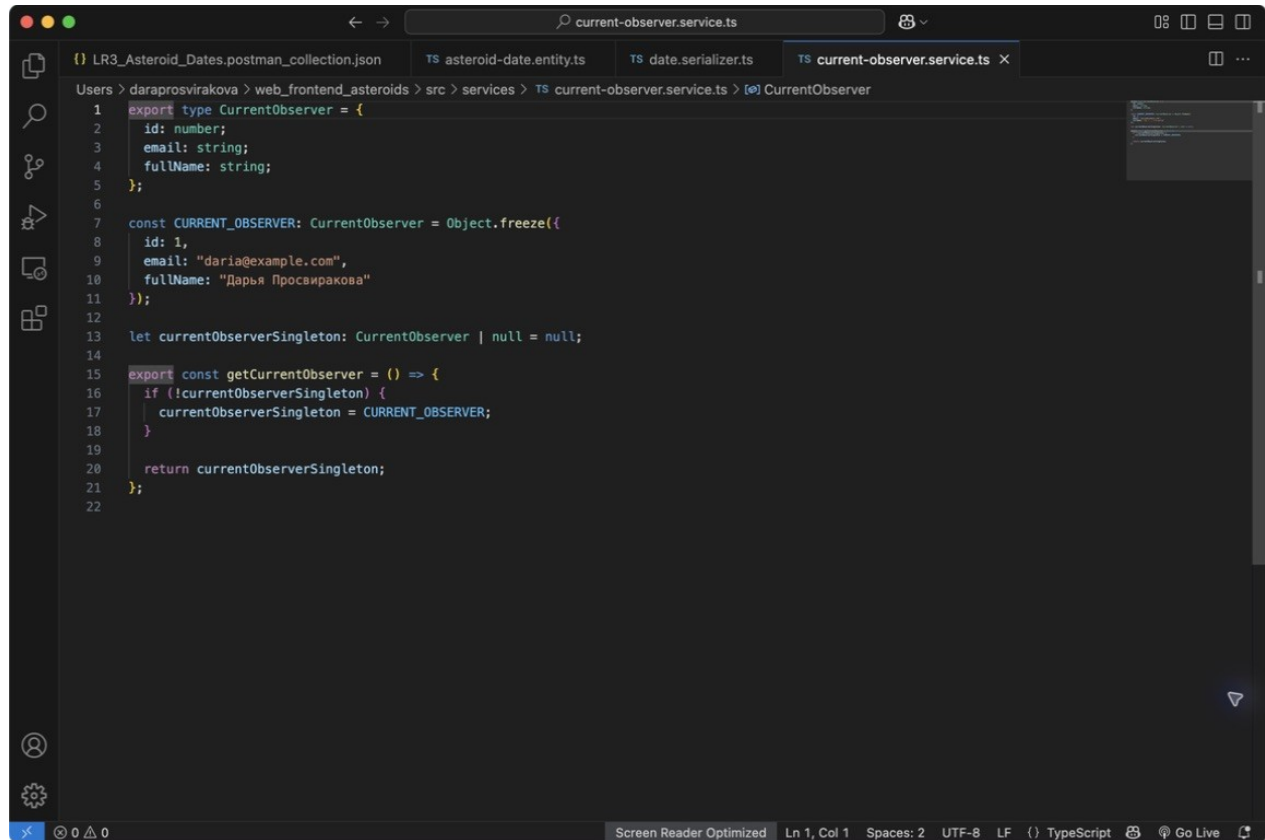
```
1 import type { AsteroidDate } from "../entities/asteroid-date.entity.js";
2 import type { DatesService } from "../services/dates.service.js";
3 import { getCurrentObserver } from "../services/current-observer.service.js";
4
5 export const serializeAsteroidDate = (date: AsteroidDate, datesService: DatesService) => {
6   const currentObserver = getCurrentObserver();
7
8   return {
9     id: date.id,
10    designation: date.designation,
11    shortDescription: date.shortDescription,
12    imageUrl: datesService.getImageUrl(date),
13    videoUrl: datesService.getVideoUrl(date),
14    approachMonth: date.approachMonth,
15    approachDay: date.approachDay,
16    minimumDistanceAu: date.minimumDistanceAu,
17    likesCount: datesService.getLikeCount(date),
18    isCreator: date.creatorId === currentObserver.id ? 1 : 0,
19    isLiked: datesService.isLikedByCurrentObserver(date) ? 1 : 0
20  };
21 };
22
```

Screen Reader Optimized Ln 1, Col 1 Spaces: 2 UTF-8 LF () TypeScript Go Live

Singleton пользователя и README

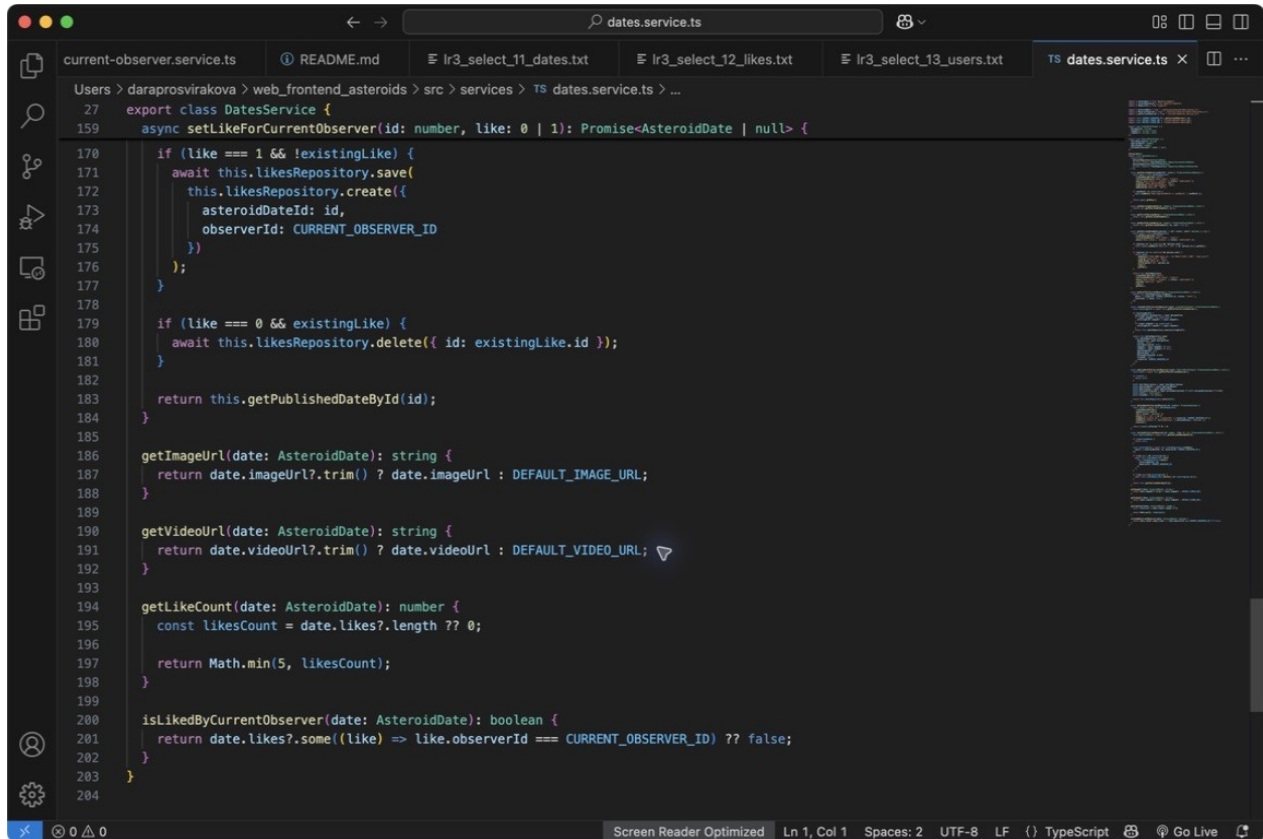
Авторизация в лабораторной не реализуется, поэтому текущий создатель зафиксирован константой в singleton-функции. Сервисные методы получают id пользователя через эту функцию и не принимают creator_id с клиента.

Скриншот 16. Singleton текущего пользователя getCurrentObserver().



```
1 export type CurrentObserver = {
2   id: number;
3   email: string;
4   fullName: string;
5 };
6
7 const CURRENT_OBSERVER: CurrentObserver = Object.freeze({
8   id: 1,
9   email: "daria@example.com",
10  fullName: "Дарья Просвиракова"
11 });
12
13 let currentObserverSingleton: CurrentObserver | null = null;
14
15 export const getCurrentObserver = () => {
16   if (!currentObserverSingleton) {
17     currentObserverSingleton = CURRENT_OBSERVER;
18   }
19   return currentObserverSingleton;
20 };
21
22
```

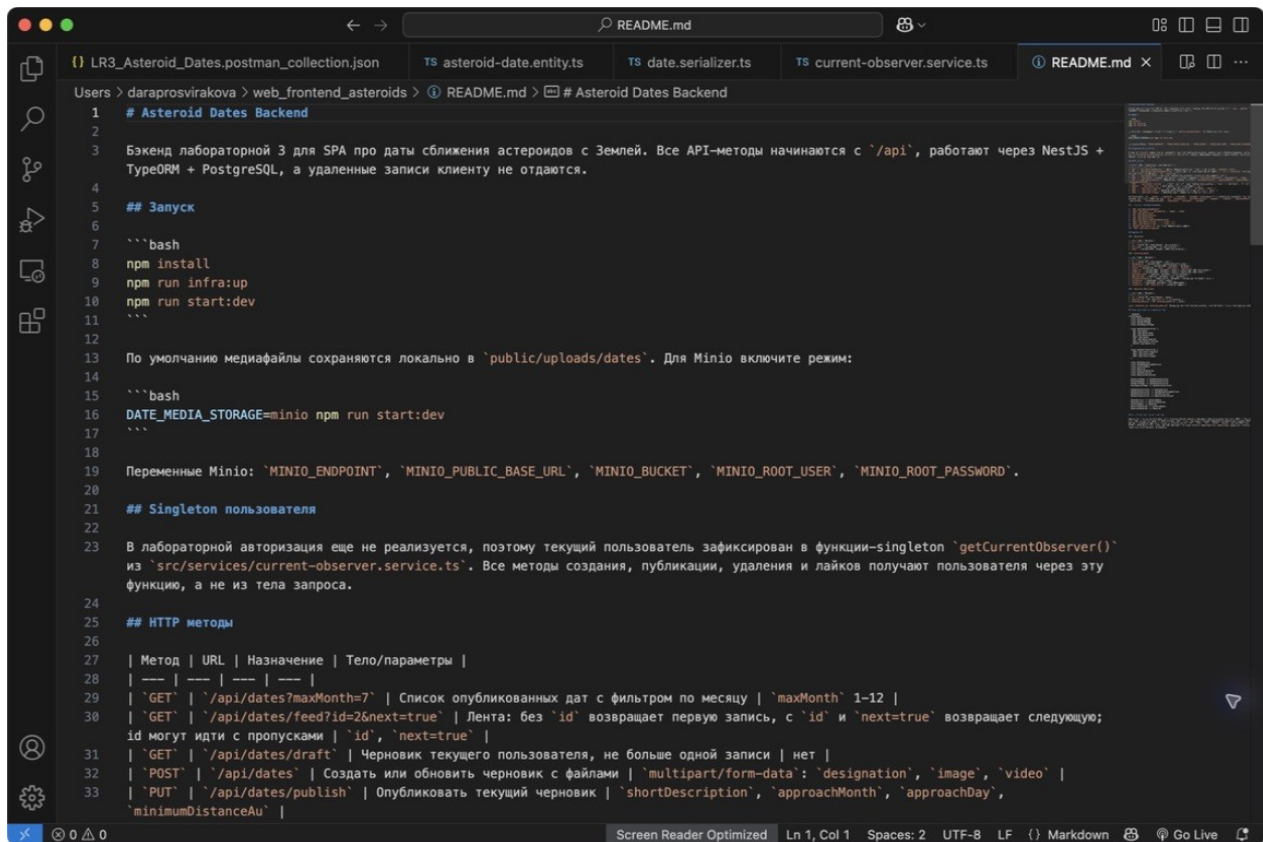
Скриншот 16.1. Использование CURRENT_OBSERVER_ID в методах сервиса и ограничение лайков до 5.



```
current-observer.service.ts
README.md
lr3_select_11_dates.txt
lr3_select_12_likes.txt
lr3_select_13_users.txt
Ts dates.service.ts

Users > daraprovirakova > web_frontend_asteroids > src > services > Ts dates.service.ts > ...
27 export class DatesService {
159   async setLikeForCurrentObserver(id: number, like: 0 | 1): Promise<AsteroidDate | null> {
170     if (like === 1 && !existingLike) {
171       await this.likesRepository.save(
172         this.likesRepository.create({
173           asteroidDateId: id,
174           observerId: CURRENT_OBSERVER_ID
175         })
176       );
177     }
178
179     if (like === 0 && existingLike) {
180       await this.likesRepository.delete({ id: existingLike.id });
181     }
182
183     return this.getPublishedDateById(id);
184   }
185
186   getImageUrl(date: AsteroidDate): string {
187     return date.imageUrl?.trim() ? date.imageUrl : DEFAULT_IMAGE_URL;
188   }
189
190   getVideoUrl(date: AsteroidDate): string {
191     return date.videoUrl?.trim() ? date.videoUrl : DEFAULT_VIDEO_URL;
192   }
193
194   getLikeCount(date: AsteroidDate): number {
195     const likesCount = date.likes?.length ?? 0;
196
197     return Math.min(5, likesCount);
198   }
199
200   isLikedByCurrentObserver(date: AsteroidDate): boolean {
201     return date.likes?.some((like) => like.observerId === CURRENT_OBSERVER_ID) ?? false;
202   }
203 }
204
```

Скриншот 17. README с запуском, HTTP методами и описанием таблиц.



```
{ LR3_Asteroid_Dates.postman_collection.json
Ts asteroid-date.entity.ts
Ts date.serializer.ts
Ts current-observer.service.ts
README.md

Users > daraprovirakova > web_frontend_asteroids > README.md > # Asteroid Dates Backend
1 # Asteroid Dates Backend
2
3 Бэкэнд лабораторной 3 для SPA про даты сближения астероидов с Землей. Все API-методы начинаются с '/api', работают через NestJS +
TypeORM + PostgreSQL, а удаленные записи клиенту не отдаются.
4
5 ## Запуск
6
7 ```bash
8 npm install
9 npm run infra:up
10 npm run start:dev
11 ```
12
13 По умолчанию медиафайлы сохраняются локально в 'public/uploads/dates'. Для Minio включите режим:
14
15 ```bash
16 DATE_MEDIA_STORAGE=minio npm run start:dev
17 ```
18
19 Переменные Minio: 'MINIO_ENDPOINT', 'MINIO_PUBLIC_BASE_URL', 'MINIO_BUCKET', 'MINIO_ROOT_USER', 'MINIO_ROOT_PASSWORD'.
20
21 ## Singleton пользователя
22
23 В лабораторной авторизация еще не реализуется, поэтому текущий пользователь зафиксирован в функции-singleton 'getCurrentObserver()'
из 'src/services/current-observer.service.ts'. Все методы создания, публикации, удаления и лайков получают пользователя через эту
функцию, а не из тела запроса.
24
25 ## HTTP методы
26
27 | Метод | URL | Назначение | Тело/параметры |
28 | --- | --- | --- | --- |
29 | 'GET' | '/api/dates?maxMonth=7' | Список опубликованных дат с фильтром по месяцу | 'maxMonth' 1-12 |
30 | 'GET' | '/api/dates/feed?id=2&next=true' | Лента: без 'id' возвращает первую запись, с 'id' и 'next=true' возвращает следующую;
id могут идти с пропусками | 'id', 'next=true' |
31 | 'GET' | '/api/dates/draft' | Черновик текущего пользователя, не больше одной записи | нет |
32 | 'POST' | '/api/dates' | Создать или обновить черновик с файлами | 'multipart/form-data': 'designation', 'image', 'video' |
33 | 'PUT' | '/api/dates/publish' | Опубликовать текущий черновик | 'shortDescription', 'approachMonth', 'approachDay',
'minimumDistanceAu' |
```

Контрольные вопросы

Веб сервис: Программный интерфейс, доступный по сети. В работе сервис отдает и изменяет данные SPA через /api.

REST: Архитектурный стиль, где ресурсы представлены URL, а действия выражаются стандартными HTTP методами.

RPC: Подход, в котором клиент вызывает удаленную процедуру как функцию, например publishDate или setLike.

HTTP заголовки и методы: Методы GET, POST, PUT, DELETE задают действие, а заголовки описывают формат, авторизацию, кэширование и служебные параметры запроса.

Версии HTTP: HTTP/1.0 и HTTP/1.1 используют текстовый протокол поверх TCP; HTTP/2 добавляет мультиплексирование; HTTP/3 работает поверх QUIC.

HTTPS: HTTP поверх TLS. Шифрует трафик, проверяет сертификат сервера и защищает данные от подмены в канале.

OSI ISO: Семислойная модель: физический, канальный, сетевой, транспортный, сеансовый, представления, прикладной уровни.